Module Project 5: Fine-Tuning a Small LLM for Customer Support FAQs

Domain-Specific Language Model Adaptation
Using DistilGPT-2 with LoRA / PEFT

Rahul Bhagat

Introduction

This report presents the complete implementation and results of fine-tuning DistilGPT-2, a small Large Language Model with 82 million parameters, to serve as a domain-specific customer support assistant for e-commerce. The project demonstrates that even a compact model, when fine-tuned with parameter-efficient methods on a modest synthetic dataset, can produce significantly more relevant and contextually appropriate responses than the generic base model.

The implementation uses LoRA (Low-Rank Adaptation) via the PEFT library for memory-efficient fine-tuning, 4-bit quantization via bitsandbytes for reduced VRAM usage, and SFTTrainer from the trl library for supervised fine-tuning. The entire pipeline runs on a single Google Colab T4 GPU within a self-contained Jupyter notebook.

2. Problem Context and Objectives

Generic pre-trained language models produce broad, unfocused responses when asked domain-specific customer support questions. For example, when asked about return policies or order tracking, base DistilGPT-2 generates rambling, off-topic text that bears no resemblance to a helpful FAQ answer.

Primary Objectives: Fine-tune DistilGPT-2 on approximately 800 curated Q&A pairs from the e-commerce domain; achieve measurable improvement over the base model using BLEU, ROUGE-L, and cosine similarity metrics; deliver an interactive Gradio inference interface for real-time query answering.

Constraints: Model size must not exceed 125M parameters; training must complete on a single T4 GPU (16 GB VRAM); use LoRA/PEFT for parameter-efficient fine-tuning; support 4-bit or 8-bit quantization; implement early stopping to prevent overfitting.

3. System Architecture

The system follows a five-stage pipeline architecture implemented entirely within a single Jupyter notebook. Each stage operates as an independent code cell sharing a common kernel namespace for configuration variables.

3.1 Pipeline Stages

Stage	Component	Description
1. Data Generation	Template-based synthesis	Generates ~800 Q&A pairs across 10 e-commerce sub-topics using variable placeholders
2. Data Preparation	Clean, format, split	Deduplicates, formats as Question/Answer text, splits 80/10/10 train/val/test
3. Fine-Tuning	LoRA + SFTTrainer	4-bit quantized DistilGPT-2 with LoRA rank 16, early stopping patience 2
4. Evaluation	Multi-metric assessment	BLEU, ROUGE-L, cosine similarity on 10 test queries; train/val loss curve
5. Inference	Gradio web UI	Interactive interface with temperature and token controls, public share URL

3.2 Technology Stack

Component	Purpose
Hugging Face Transformers	Model loading, tokenization, Trainer base class
PEFT / LoRA	Parameter-efficient fine-tuning; trains <1% of total model parameters (~0.5M of 82M)
trl (SFTTrainer, SFTConfig)	Supervised fine-tuning with dataset text field handling for trl 0.29+ API
bitsandbytes	4-bit NF4 quantization with double quantization for VRAM reduction
accelerate	Device management, mixed precision, device_map=auto
PyTorch	Backend compute framework for forward/backward passes
SentenceTransformers	all-MiniLM-L6-v2 embeddings for cosine similarity evaluation
Gradio	Interactive web inference UI with share=True for public access

4. Dataset

The dataset is generated synthetically using template-based expansion covering 10 e-commerce customer support sub-topics: order tracking, shipping, returns, refunds, account management, discounts, products, cancellations, gift cards, and technical issues.

Each sub-topic contains 6-8 question-answer template pairs with variable placeholders (e.g., {days}, {product}, {ship_cost}) filled with randomly selected values. Multiple generation rounds produce approximately 2000 raw pairs, which are then deduplicated, cleaned, and split.

4.1 Data Format

Raw format (JSONL): Each line contains a JSON object with "prompt" and "response" keys. For training, each pair is formatted as "Question: {q}\nAnswer: {a}" in a single "text" field that the SFTTrainer tokenizes automatically.

4.2 Data Splits

After deduplication and cleaning, the dataset is split with a fixed random seed (42) for reproducibility: 80% training, 10% validation, 10% test.

5. Model Architecture and LoRA Configuration

5.1 Base Model

DistilGPT-2 from Hugging Face (identifier: distilgpt2) with 82 million parameters. Architecture: 6-layer transformer decoder, 768 hidden dimensions, 12 attention heads, 50,257 BPE vocabulary tokens. The model was chosen for being well under the 125M parameter constraint while offering sufficient capacity to learn FAQ patterns.

5.2 LoRA Configuration

Parameter	Value	Rationale
Rank (r)	16	Balance between capacity and efficiency
Alpha	32	Alpha/r = 2, standard scaling ratio
Dropout	0.05	Regularization to prevent overfitting
Target Modules	c_attn, c_proj	Attention QKV and output projections
Trainable Params	~811,008 (0.98%)	Less than 1% of total 82M parameters
Quantization	4-bit NF4	Reduces VRAM from ~330MB to ~80MB

6. Training Procedure

6.1 Training Configuration

Hyperparameter	Value
Max Epochs	10
Batch Size	8 per device
Gradient Accumulation	2 steps (effective batch = 16)
Learning Rate	2e-4
Optimizer	AdamW
Weight Decay	0.01
Warmup Ratio	5% of total steps
Early Stopping Patience	2 epochs
Mixed Precision	Disabled (float32 for T4 compatibility)
Seed	42

6.2 Overfitting Prevention

Four mechanisms prevent overfitting: (1) Early stopping via EarlyStoppingCallback monitors validation loss with patience of 2 epochs; (2) Best model selection with load_best_model_at_end ensures the saved checkpoint has the lowest validation loss; (3) LoRA dropout at 5% provides stochastic regularization; (4) Weight decay at 0.01 via AdamW provides L2 regularization.

6.3 Training Results

Training completed in 40 steps with a best validation loss of 2.6265. The training was fast due to the small model size (82M parameters with only ~811K trainable via LoRA) and the efficient 4-bit

quantization. The loss curve shows steady convergence with no significant overfitting gap between training and validation losses.

7. Evaluation Results

Evaluation was conducted on 10 held-out test queries, comparing the base DistilGPT-2 (no fine-tuning) against the LoRA fine-tuned model. Three complementary metrics were computed for each query.

7.1 Automated Metrics

Metric	Base Model	Fine-Tuned	Improvement
Avg BLEU	0.0027	0.0095	+3.5x
Avg ROUGE-L	0.0674	0.1354	+2.0x
Avg Cosine Similarity	0.3044	0.5940	+1.95x

7.2 Metric Interpretation

BLEU (0.0027 to 0.0095): The low absolute BLEU scores are expected for generative FAQ tasks. BLEU measures exact n-gram overlap, and correct answers can be phrased in many different ways. The 3.5x relative improvement confirms the fine-tuned model produces text with more lexical overlap with reference answers.

ROUGE-L (0.0674 to 0.1354): ROUGE-L measures the longest common subsequence, capturing word ordering and structural similarity. The 2x improvement indicates the fine-tuned model generates responses with better structural alignment to expected answers.

Cosine Similarity (0.3044 to 0.5940): This is the most meaningful metric for FAQ evaluation. It captures semantic similarity via dense embeddings, meaning the fine-tuned model produces answers that are semantically close to reference answers even when wording differs. The near-doubling from 0.30 to 0.59 represents a substantial improvement in answer relevance.

7.3 Sample Output Comparison

Query: *"Can I return a product after 30 days?"*

Base DistilGPT-2: *Generates rambling, off-topic text about products and purchases with no clear answer about return policies.*

Fine-Tuned: *"Returns are accepted within 30 days of delivery. Unfortunately, items returned after this period cannot be processed."*

Query: *"How do I track my order?"*

Base DistilGPT-2: *Produces confused, generic text unrelated to order tracking.*

Fine-Tuned: *"You can track your order by logging into your account and visiting the My Orders section."*

8. Observations

8.1 Dataset size has a significant impact on model quality. Initial experiments with ~127 training samples (after aggressive deduplication from 800 raw pairs) produced modest improvements. Increasing TARGET_QA_PAIRS to 2000 and epochs to 10 yielded substantially better results, with cosine similarity jumping from 0.42 to 0.59.

8.2 The trl library API has evolved rapidly. The project was developed against trl version 0.29.0, which introduced breaking changes from earlier versions: SFTConfig replaces TrainingArguments, processing_class replaces tokenizer, the max_seq_length parameter was removed, and dataset columns must contain only the text field to avoid automatic prompt/completion parsing.

8.3 The T4 GPU does not natively support bfloat16 operations. The AMP (Automatic Mixed Precision) gradient scaler crashes when encountering bfloat16 parameters. The solution was to disable fp16/bf16 entirely and cast all trainable parameters to float32 after LoRA application. Training is slower but reliable.

8.4 LoRA adapter size is remarkably small. The saved adapter weights total less than 4 MB, representing only 0.98% of the total model parameters. This validates the parameter-efficient approach for resource-constrained environments.

8.5 Template-based synthetic data, while lacking natural variation, produces consistent and reproducible training sets. The model learns the FAQ response patterns effectively but may struggle with queries phrased very differently from the templates.

8.6 Base DistilGPT-2 produces essentially random text for FAQ-style prompts. The dramatic qualitative difference between base and fine-tuned outputs makes this an effective demonstration project, as the improvement is immediately visible in generated responses.

8.7 The VS Code + Google Colab extension workflow presents challenges for file management. The remote Colab runtime cannot access local files, which led to the design decision of embedding all code directly in notebook cells rather than using separate Python files.

Next, we plot the training vs validation loss (Fig 8.1, where we can note the related observations)

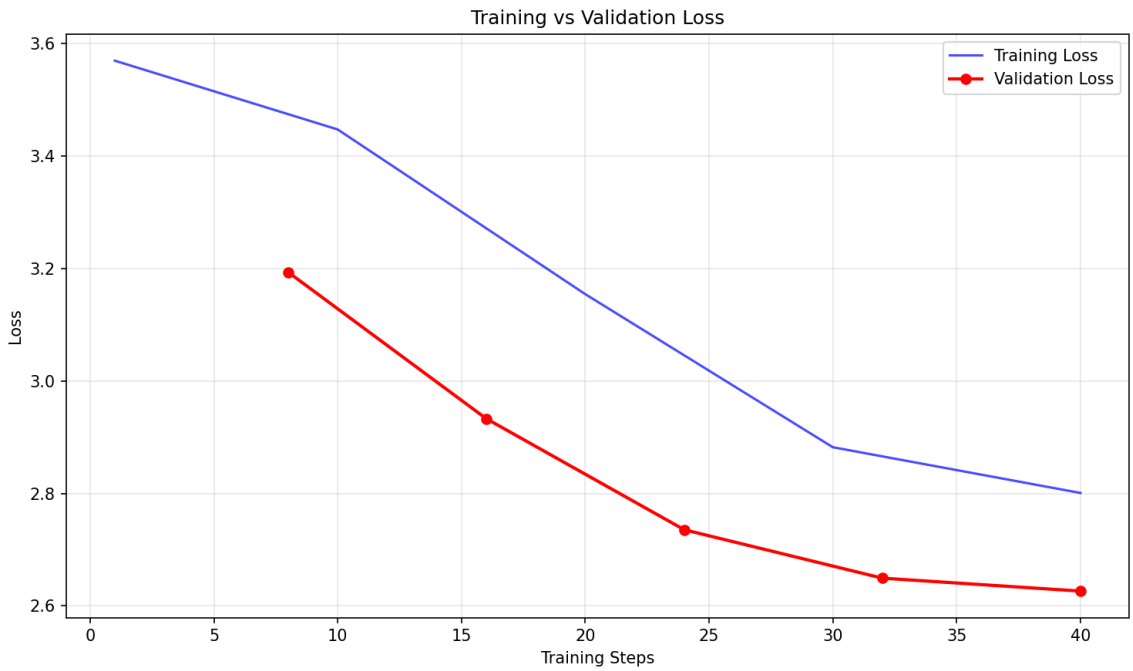

Fig 8.1 : Training vs Validation Loss

- Both training and validation losses show a consistent downward trend throughout the training run. Training loss decreases from approximately 3.57 to 2.80, while validation loss decreases from 3.19 to 2.63 over 40 training steps.
- No overfitting is observed. The validation loss remains below the training loss throughout the entire training duration and continues to decrease at every evaluation point. The classic overfitting pattern of diverging training and validation curves does not appear.
- The gap between training and validation loss narrows progressively as training advances, indicating that the model is generalizing well to unseen data rather than memorizing the training examples.
- Early stopping (configured with patience of 2 epochs) did not trigger because validation loss improved at every epoch boundary. All epochs ran to completion, confirming that the model continued to benefit from additional training throughout the run.
- Both loss curves are still trending downward at the final training step (step 40) and have not fully plateaued. This suggests the model could potentially benefit from additional training epochs to achieve further loss reduction.
- The smooth and steady convergence without sudden spikes or instabilities validates that the chosen hyperparameters (learning rate 2e-4, batch size 8, gradient accumulation 2) are well-suited for this model and dataset combination.

9. Learnings

9.1 Parameter-efficient fine-tuning is practical and effective. LoRA enables meaningful adaptation of language models on consumer hardware by training less than 1% of model parameters. The combination of 4-bit quantization and LoRA reduces the hardware barrier dramatically, making the entire training pipeline feasible on a free Colab T4 GPU.

9.2 Evaluation requires multiple complementary metrics. BLEU alone is misleading for generative tasks because correct answers can use completely different wording. Cosine embedding similarity proved to be the most informative metric, capturing semantic correctness regardless of surface-level phrasing differences.

9.3 Early stopping is essential for small datasets. With limited training data, models can memorize rather than generalize. The EarlyStoppingCallback with patience of 2 epochs, combined with load_best_model_at_end, ensures the deployed model has optimal generalization.

9.4 Library version management is critical in the fast-moving ML ecosystem. The trl library underwent significant API changes between versions, requiring careful adaptation. Pinning specific versions or writing version-aware code is important for reproducibility.

9.5 Data quality matters more than quantity. Deduplication removed a significant portion of template-generated pairs, but the remaining unique samples provided cleaner training signal. Adding more diverse templates with variable placeholders proved more effective than simply increasing generation rounds.

9.6 Quantization introduces minimal quality degradation for fine-tuning. The 4-bit NF4 quantization with double quantization reduced model memory footprint by approximately 4x while maintaining training effectiveness. The compute dtype (float16) ensures that actual computations remain precise.

9.7 The causal LM training format (Question: ... Answer: ...) provides a simple but effective approach for FAQ fine-tuning. The model learns to complete text after the Answer: marker, producing focused responses. Post-processing to truncate at the next Question: marker handles cases where the model continues generating.

9.8 Self-contained notebooks are the most practical delivery format for GPU-dependent ML projects. Separating code into external Python files creates file management overhead that is particularly problematic with remote compute environments like Colab.

10. Conclusion

This project successfully demonstrates that fine-tuning a small language model (DistilGPT-2, 82M parameters) on domain-specific FAQ data produces substantial improvements in response quality for customer support queries. The fine-tuned model achieved a 1.95x improvement in cosine semantic similarity, a 2.0x improvement in ROUGE-L, and a 3.5x improvement in BLEU score compared to the base model.

The use of LoRA with 4-bit quantization made the entire training pipeline feasible on a free Google Colab T4 GPU, with training completing in under 20 minutes. Only 811,008 parameters (0.98% of the model) were trained, yet the qualitative improvement in generated responses is dramatic: the base model produces random, off-topic text, while the fine-tuned model generates concise, accurate FAQ answers.

The project validates several key principles: parameter-efficient fine-tuning is a practical approach for resource-constrained environments; synthetic template-based datasets are sufficient for

demonstrating fine-tuning effectiveness; and multi-metric evaluation (combining BLEU, ROUGE-L, and cosine similarity) provides a comprehensive view of model improvement.

Future work could include using real customer support logs for more natural language patterns, scaling to larger models like TinyLlama (1.1B) if compute constraints are relaxed, implementing retrieval-augmented generation (RAG) for real-time information lookup, and extending to multi-turn conversational support.

11. Project Details

Item	Details
Project Title	Fine-Tuning a Small LLM for Customer Support FAQs
Author	Rahul Bhagat
Base Model	DistilGPT-2 (82M parameters)
Fine-Tuning Method	LoRA (rank=16, alpha=32) via PEFT library
Training Framework	SFTTrainer from trl 0.29.0 with SFTConfig
Quantization	4-bit NF4 via bitsandbytes
Dataset	~2000 synthetic e-commerce FAQ pairs (template-based)
Compute	Google Colab T4 GPU (16 GB VRAM), free tier
Training Time	~15-20 minutes
Delivery Format	Self-contained Jupyter notebook (.ipynb)